

Program 1: Singly Linked List: Insertion, Deletion and Display

Aim

To write a C program that performs insertion (at beginning, end, position), deletion (from beginning, end, position) and display on a singly linked list.

Theory

A linked list node holds a **data** field and a **next** pointer; **head** points to the first node and the last node points to NULL. Nodes are allocated dynamically with `malloc()` and released with `free()`.

Insertion: at the beginning is $O(1)$ (new node points to old head; head = new node). At the end or at a position it is $O(n)$ because the list must be traversed to reach the previous node.

Deletion: the pointer of the previous node is made to skip the target node, and the node is freed. Deleting the first node just moves head forward.

Unlike arrays, no elements are shifted - only pointers are changed. Positions are 1-based.

Algorithm

1. Create a node with malloc and set its data and next fields.
2. Insert at beginning: `new->next = head; head = new`.
3. Insert at end / position: traverse to the last node / the node before the position; `new->next = temp->next; temp->next = new`.
4. Delete from beginning: `temp = head; head = head->next; free(temp)`.
5. Delete from end / position: find the previous node; link it to the node after the target; free the target.
6. Display: traverse from head printing each data value until NULL.
7. Repeat through a menu until Exit; free all remaining nodes.

C Source Code

```
/*
 * Program : Singly Linked List - Insertion, Deletion and Display
 *
 * THEORY:
 * - A linked list is a linear data structure made of NODES. Each node has
 *   a data part and a pointer (next) to the following node. The last node
 *   points to NULL. Nodes are created dynamically with malloc().
 * - "head" always points to the first node.
 * - Insertion : at beginning  $O(1)$ , at end  $O(n)$ , at a position  $O(n)$ .
 * - Deletion : from beginning  $O(1)$ , from end  $O(n)$ , from a position  $O(n)$ .
 * - No shifting of elements is needed (unlike arrays) - only pointers change.
 * - Positions are 1-based (1 = first node).
 */
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL;

struct Node *createNode(int val) {
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
```

```

        n->data = val;
        n->next = NULL;
        return n;
    }

void insertBegin(int val) {
    struct Node *n = createNode(val);
    n->next = head;
    head = n;
    printf("Inserted %d at beginning.\n", val);
}

void insertEnd(int val) {
    struct Node *n = createNode(val);
    if (head == NULL) head = n;
    else {
        struct Node *t = head;
        while (t->next != NULL) t = t->next;
        t->next = n;
    }
    printf("Inserted %d at end.\n", val);
}

void insertPos(int pos, int val) {
    if (pos < 1) { printf("Invalid position!\n"); return; }
    if (pos == 1) { insertBegin(val); return; }
    struct Node *t = head;
    for (int i = 1; i < pos - 1 && t != NULL; i++) t = t->next;
    if (t == NULL) { printf("Invalid position!\n"); return; }
    struct Node *n = createNode(val);
    n->next = t->next;
    t->next = n;
    printf("Inserted %d at position %d.\n", val, pos);
}

void deleteBegin() {
    if (head == NULL) { printf("List is empty!\n"); return; }
    struct Node *t = head;
    head = head->next;
    printf("Deleted %d from beginning.\n", t->data);
    free(t);
}

void deleteEnd() {
    if (head == NULL) { printf("List is empty!\n"); return; }
    if (head->next == NULL) {
        printf("Deleted %d from end.\n", head->data);
        free(head);
        head = NULL;
        return;
    }
    struct Node *t = head;
    while (t->next->next != NULL) t = t->next; /* second last node */
    printf("Deleted %d from end.\n", t->next->data);
    free(t->next);
    t->next = NULL;
}

void deletePos(int pos) {
    if (head == NULL) { printf("List is empty!\n"); return; }
    if (pos < 1) { printf("Invalid position!\n"); return; }
    if (pos == 1) { deleteBegin(); return; }
    struct Node *t = head;
    for (int i = 1; i < pos - 1 && t != NULL; i++) t = t->next;
    if (t == NULL || t->next == NULL) { printf("Invalid position!\n"); return; }
    struct Node *del = t->next;
    t->next = del->next;
    printf("Deleted %d from position %d.\n", del->data, pos);
    free(del);
}

void display() {

```

```

        if (head == NULL) { printf("List is empty.\n"); return; }
        printf("List: ");
        for (struct Node *t = head; t != NULL; t = t->next)
            printf("%d -> ", t->data);
        printf("NULL\n");
    }

int main() {
    int choice, val, pos;
    do {
        printf("\n1.Insert Begin  2.Insert End  3.Insert Position\n"
              "4.Delete Begin  5.Delete End  6.Delete Position\n"
              "7.Display      8.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("Enter value: ");          scanf("%d", &val);          insertBegin(val); break;
            case 2: printf("Enter value: ");          scanf("%d", &val);          insertEnd(val);   break;
            case 3: printf("Enter position and value: "); scanf("%d %d", &pos, &val); insertPos(pos, val); bre
            case 4: deleteBegin(); break;
            case 5: deleteEnd();   break;
            case 6: printf("Enter position: ");        scanf("%d", &pos);        deletePos(pos);   break;
            case 7: display();    break;
            case 8: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 8);

    while (head != NULL) { /* free remaining nodes */
        struct Node *t = head; head = head->next; free(t);
    }
    return 0;
}

```

Sample Input

Menu choices: Insert End 10, Insert End 20, Insert Begin 5, Insert Position 3 value 15, Display, Delete Begin, Delete End, Display, Delete Position 5 (invalid), Display, Exit.

Sample Output (program run with the input above)

```

1.Insert Begin  2.Insert End  3.Insert Position
4.Delete Begin  5.Delete End  6.Delete Position
7.Display      8.Exit
Enter choice: Enter value: Inserted 10 at end.

1.Insert Begin  2.Insert End  3.Insert Position
4.Delete Begin  5.Delete End  6.Delete Position
7.Display      8.Exit
Enter choice: Enter value: Inserted 20 at end.

1.Insert Begin  2.Insert End  3.Insert Position
4.Delete Begin  5.Delete End  6.Delete Position
7.Display      8.Exit
Enter choice: Enter value: Inserted 5 at beginning.

1.Insert Begin  2.Insert End  3.Insert Position
4.Delete Begin  5.Delete End  6.Delete Position
7.Display      8.Exit
Enter choice: Enter position and value: Inserted 15 at position 3.

1.Insert Begin  2.Insert End  3.Insert Position
4.Delete Begin  5.Delete End  6.Delete Position
7.Display      8.Exit
Enter choice: List: 5 -> 10 -> 15 -> 20 -> NULL

1.Insert Begin  2.Insert End  3.Insert Position
4.Delete Begin  5.Delete End  6.Delete Position

```

```

7.Display      8.Exit
Enter choice: Deleted 5 from beginning.

1.Insert Begin 2.Insert End 3.Insert Position
4.Delete Begin 5.Delete End 6.Delete Position
7.Display      8.Exit
Enter choice: Deleted 20 from end.

1.Insert Begin 2.Insert End 3.Insert Position
4.Delete Begin 5.Delete End 6.Delete Position
7.Display      8.Exit
Enter choice: List: 10 -> 15 -> NULL

1.Insert Begin 2.Insert End 3.Insert Position
4.Delete Begin 5.Delete End 6.Delete Position
7.Display      8.Exit
Enter choice: Enter position: Invalid position!

1.Insert Begin 2.Insert End 3.Insert Position
4.Delete Begin 5.Delete End 6.Delete Position
7.Display      8.Exit
Enter choice: List: 10 -> 15 -> NULL

1.Insert Begin 2.Insert End 3.Insert Position
4.Delete Begin 5.Delete End 6.Delete Position
7.Display      8.Exit
Enter choice: Exiting...

```